

Condition Variable in C++

What is a Condition Variable?

A condition variable in C++ is a synchronization primitive used to coordinate threads. One thread can wait for a certain condition to become true while another thread notifies it when the condition is met.

Real-World Analogy

Imagine a waiter (thread A) waiting for the chef (thread B) to finish cooking. The waiter doesn't constantly check the kitchen; he waits to be notified when the food is ready.

Core Components

- `std::mutex`: to protect shared data.
- `std::condition_variable`: for thread signaling.
- `std::unique_lock<std::mutex>`: required for `condition_variable::wait`.

Basic Example: Producer-Consumer

```
#include <iostream>

#include <thread>

#include <condition_variable>

#include <mutex>

using namespace std;

mutex mtx;

condition_variable cv;

bool ready = false;

void consumer() {

    unique_lock<mutex> lock(mtx);

    cout << "Consumer waiting..." << endl;
```

Condition Variable in C++

```
cv.wait(lock, [] { return ready; });  
cout << "Consumer proceeding!" << endl;  
}  
  
void producer() {  
    this_thread::sleep_for(chrono::seconds(2));  
    {  
        lock_guard<mutex> lock(mtx);  
        ready = true;  
        cout << "Producer setting ready = true" << endl;  
    }  
    cv.notify_one();  
}  
  
int main() {  
    thread t1(consumer);  
    thread t2(producer);  
    t1.join();  
    t2.join();  
    return 0;  
}
```

Explanation

- consumer() waits for the condition variable until 'ready' becomes true.
- producer() sets 'ready' and notifies one waiting thread using notify_one().
- The consumer resumes execution once notified.

Why Use Condition Variables?

- Avoid busy-waiting (inefficient CPU usage).

Condition Variable in C++

- Efficient and safe thread synchronization.

Key Functions of `std::condition_variable`

- `wait(lock)`: blocks until notified.
- `wait(lock, pred)`: blocks until `pred` returns true.
- `notify_one()`: wakes one waiting thread.
- `notify_all()`: wakes all waiting threads.